

Quantifying Novice Behavior, Experience, and Mental Effort in Code Puzzle Pathways

John Allen

Washington University in St. Louis
St. Louis, MO, USA
johnjallen@wustl.edu

Caitlin Kelleher

Washington University in St. Louis
St. Louis, MO, USA
ckelleher@wustl.edu

ABSTRACT

Code puzzles are an increasingly popular approach to introducing programming to young learners. Today, code puzzles are predominantly introduced through static puzzle sequences with increasing difficulty. However, adaptive systems in other domains have improved learning efficiency. This paper takes a step towards developing adaptive code puzzle systems based on controlling learners' cognitive load. We conducted a study comparing static code puzzle pathways and adaptive pathways that predict cognitive load on future puzzles. While the trialled adaptive recommendation policy did not result in better learning, our findings point us towards a different policy which may have a greater effect on learner experience. In addition, we identify predictors of student dropout, and use our experimental data to quantify learners' puzzle-solving experiences into 7 principal component properties and use these factors to suggest approaches for future adaptive systems.

CCS CONCEPTS

• **Human-centered computing** → **Human-computer interaction (HCI)**.

KEYWORDS

Code Puzzles, Adaptive Learning Systems, Cognitive Load

ACM Reference Format:

John Allen and Caitlin Kelleher. 2021. Quantifying Novice Behavior, Experience, and Mental Effort in Code Puzzle Pathways. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3411763.3451752>

1 INTRODUCTION

An increasing number of young students are introduced to programming through online resources such as tutorials, games, and code puzzles. Code puzzles are a popular and promising way of introducing young students to programming [12]. In blocks-based code puzzles, novices are typically given a set of code blocks which correspond with animations that the user must correctly assemble to recreate an action or scene [1–3]. Progressively more difficult code

puzzles can be arranged in a sequence or “pathway” to introduce programming constructs to novices [1–3].

Code puzzles are usually introduced in fixed sequences, however, adaptive learning systems have been shown to benefit learning in other contexts. Adaptive systems could potentially recommend future code puzzles which best suit an individual learner.

Some adaptive systems have shown promise personalizing learning by controlling a learner's cognitive load [9, 23, 29]. Cognitive load can be measured [27] and managed to improve learning [31, 32, 34]. Furthermore, it is possible to predict cognitive load in future code puzzles [17]. Predicting cognitive load for potential future puzzles can be leveraged in an adaptive learning system to recommend future puzzles, and may be able to be used to control learners' mental effort.

We frame this experiment as an initial dosing trial for an adaptive learning system which aims to manage cognitive load to lead to more effective learning by recommending code puzzles which are best suited to a specific learner. A diagram of our system's flow is shown in Figure 2. In this study, we pilot a recommendation policy that suggests future code puzzles which are predicted to be neither too easy nor too difficult. We evaluate the trialled recommendation policy to determine its effect on learning. We also use the user data collected to piece together a picture of what trends in cognitive load and puzzle-solving behavior were indicative of success, and use these insights to discuss potential informed changes to our initial recommendation policy.

We ask three questions: 1) Does our code recommendation policy affect cognitive load or learning outcomes versus fixed puzzle sequences? 2) What range of cognitive load is indicative of successful learning experiences? And 3) What factors best summarize how learners interact with code puzzles, and how can these patterns be used to describe a puzzle-solving experience? We show that 1) our puzzle recommendation policy did not significantly change any learning outcomes versus static pathways, 2) maintaining lower cognitive loads may reduce student dropout, and 3) we identify and discuss the 7 traits which most describe how users interact within a puzzle interface.

2 BACKGROUND

Cognitive load theory suggests that optimizing cognitive load can improve learning. Adaptive learning systems have taken advantage of this theory to promote effective learning in several domains. Researchers have recently developed a variety of code puzzle systems to introduce programming with promising results, yet these puzzles are currently introduced in fixed, one-size-fits all series of tasks. This work is a step towards the development of an adaptive code puzzle system which improves learning by managing cognitive

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8095-9/21/05...\$15.00

<https://doi.org/10.1145/3411763.3451752>

load, and builds upon previous work in cognitive load theory, code puzzles, and adaptive learning systems.

2.1 Cognitive Load Theory

Cognitive Load Theory (CLT) is a theory of instructional design which emphasizes the relationship between working memory and long-term memory and their impacts on effective learning. Working memory is the conscious processing of information, and is highly limited in both space and duration [31, 32, 34]. A concept, or novel piece of information, must be processed in working memory before being committed to long-term memory. Information can be retrieved from long-term memory and used to process and organize more novel information in working memory at no cost to working memory capacity [31, 32]. CLT suggests that working memory may be optimized to improve learning [34] by controlling three categories of load on working memory: intrinsic, germane, and extraneous loads. Intrinsic load refers to the working memory caused by the complexity of the information in a task combined with the learner's ability to complete that task [34]. As learners develop a schema for the concepts relevant to a task in long-term memory, their intrinsic load for a given task decreases, because they have to process fewer pieces of novel information in that task. For example, a simple addition question should yield a smaller intrinsic load for a college student than for a kindergartner due to the college student's expertise with addition. Germane load is associated with the working memory a learner applies to developing schema to better understand the concepts of a task, contributing to more effective learning [34]. CLT suggests maximizing germane load to promote effective learning. Finally, extraneous load is caused by complexities of a task which are irrelevant to the intended learning outcomes [34]. Extraneous load can be introduced by poor presentation of information related to a task. CLT suggests minimizing extraneous load to increase the efficiency of learning. This can be done through carefully designing tasks to avoid causes for extraneous load [22]. Intrinsic load can only be changed by changing the content of a task or by changing the expertise of a learner [32]. An effective educational tool may minimize extraneous load and progressively add to the expertise of each learner for effective learning. Recently, working memory has been shown to deplete when overworked, and replenishes after rest [10]. This may suggest the usefulness of educational content which allows for periods of low required cognitive effort in order to replenish a person's working memory availability. In general, CLT emphasizes that instructional design should be made to avoid cognitive overload [22, 31, 32].

2.2 Code Puzzles

Learning programming typically involves a high intrinsic cognitive load. CLT should be considered carefully when introducing novices to programming to not overload learners [13]. In traditional introductory programming courses, novices are required to write code from scratch, introducing extraneous cognitive load in the form of syntax errors, which can be difficult for novice programmers to fix [7]. Code puzzles naturally decrease extraneous load by eliminating the risk of syntax errors and reducing how a problem is represented. Code puzzles have been shown to yield higher learning efficiency

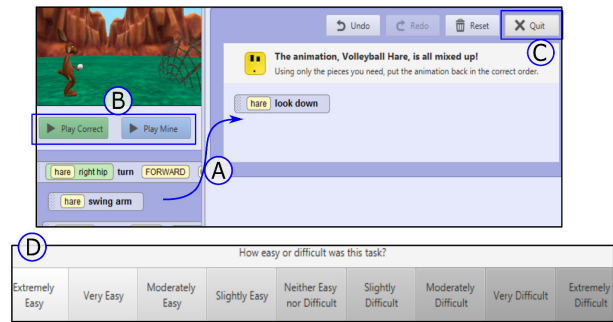


Figure 1: The code puzzle interface allows users to A) drag code blocks into their program to be executed, B) play their constructed animation as well as the reference animation, C) quit the current puzzle (all progress is lost), and finally D) record perceived mental effort upon completion of a given puzzle.

than other online methods to introduce programming, such as online tutorials [16, 20] and manually writing and debugging code [12]. Code puzzles can themselves be designed with CLT in mind. For example, to reduce extraneous load, code puzzles should not include code blocks which are not part of the solution [15]. The intrinsic load of each puzzle can also be reduced by limiting the number of elements in the puzzle, clearly segmenting the code blocks into distinct animations [30], and representing biologically natural human motor function motions in the animations [25]. The intrinsic load of code puzzles can also be managed by controlling the expertise of the learner. If a learner is prompted to solve a set of code puzzles with varying complexities, the learner's intrinsic load can be affected by the order in which they see the puzzles. This work primarily focuses on how to better progress learners through a series of code puzzles to lead to better learning.

2.3 Adaptive Learning Systems

Adaptive learning systems aim to personalize online learning systems to cater to individual users. These systems estimate a learner's expertise and provide them with content which is likely to lead to better learning. Intelligent tutoring systems (ITS) are a popular example of adaptive learning systems, and have been shown to improve student performance in many domains [19]. In order to improve learning, adaptive systems can be evaluated by their ability to interpret the knowledge and ability of the user, and determine relevant future educational content to suit a given user [8]. Adaptive learning systems include three major parts: the student performance model, a model of content complexity, and a model matching content to a student [24]. Student Modeling is used to characterize a user based on their interactions and apparent subject knowledge. For example, as students progress through tasks, knowledge acquisition can be estimated and used to categorize students [11]. In contrast to estimated knowledge acquisition, students can also be modeled based on their behaviours within an interface [6], or by their perceived cognitive load [23, 29].

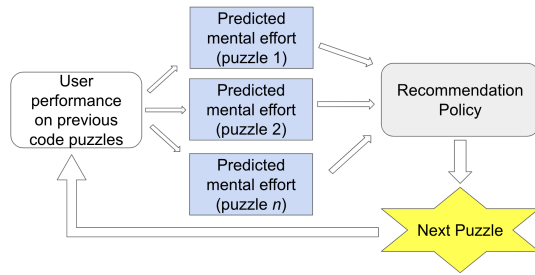


Figure 2: Our proposed adaptive code puzzle system uses user performance data to predict mental effort in future code puzzles and caters to the user’s current schema to control cognitive load.

3 METHODS

For this study we distributed two versions of the Looking Glass (LG) programming environment [2] that included code puzzle pathways to a group of young novice programmers. The control group version included static pathways of progressively more difficult puzzles, while the experimental group’s version used an adaptive puzzle recommendation system based on predicted cognitive load. We compare the performance of the two groups.

We also identify trends and behaviours shown by users using both versions of LG, and describe how the discovered behaviors relate to successful learning outcomes. The code puzzle interface is shown in more detail in Figure 1.

3.1 Adaptive Recommendation Policy

Previous research has shown that as a novice programmer works through code puzzles, random forest models can reliably use the learner’s puzzle-solving interaction data to rank unseen potential future code puzzles by mental effort (M.E.) likely to be experienced by the user [17]. Our adaptive system uses these models to rank the puzzles for each pathway for each user, and then recommends puzzles that are most likely to cause medium cognitive load (5 on M.E. likert), which is reported as “Neither too easy nor too difficult”. We chose this policy to prevent users from being either overwhelmed by difficulty or unchallenged and bored.

3.2 Data Collection Platform

LG allows users to work through code puzzles to match a “correct” animated video. Users are able to watch the correct animation, but do not have access to the completed code. Users also have the ability to run their code as they have arranged the code blocks, and compare the produced animation to the correct animation. We use LG for its code puzzle interface which is unlikely to have been used before by any of our recruited participants. We presented users with five different pathways, or series of puzzles designed to instruct specific coding constructs.

1. *Demo*: introduces sequential block programming
2. *Dotogether* includes a code construct which allows actions to be programmed in parallel
3. *Repeat* introduces the “count” construct for creating loops for repeating actions
4. *Repeat-integrative* combines “do together” and “repeat” constructs

5. *Doinorder* introduces the “do in order” construct block which allows for threads within “do together” constructs to have multiple blocks run in an order. This pathway incorporates all code constructs.

3.3 Users

We recruited a total of 127 participants for this study which included an in-person introductory workshop followed by at-home completion of code puzzles. All participants received a USB copy of LG to use at home that recorded their activity within the system. The average age of the participants was 12.89 (sd = 1.56). 65 identified as male. We received usage data back from 114.

3.4 Code Puzzle Exploratory Study

3.4.1 Data. From the 114 participants, we only analyze the 68 who complete at least the first two pathways: “demo” and “dotogether” (DoT). We do not consider those who quit before finishing these two pathways because they do not provide data from enough puzzles to produce meaningful measurement averages. These 68 users provide data from 2057 total puzzles. At the end of each puzzle, users were prompted to record their perceived M.E. on a 9-point likert scale from “extremely easy” (1) to “extremely difficult” (9). This measurement represents perceived cognitive load [27], has been widely adopted as an acceptable measurement of cognitive load, and has been verified to be as reliable as objective physiological measurements of cognitive load [33]. When recorded for each of a series of tasks, M.E. can be averaged to estimate total cognitive load throughout the tasks [26, 35]. We take the average of each user’s perceived M.E. for each pathway as a measure of their cognitive load for that pathway. For the purpose of this paper, we do not include incomplete puzzles in our cognitive load evaluation, as users often quit puzzles to look at other puzzles, rather than the puzzle itself being too difficult. As users attempted to solve each puzzle, their interactions within the interface were logged comprehensively, tracking the full user-generated program state at all times. For each pathway a user completed, the user was compensated with a \$5 gift card.

4 ANALYSIS

We reiterate our three questions: 1) Does our code recommendation policy affect cognitive load or learning outcomes? 2) What range of cognitive load is indicative of successful experiences? And 3) What factors best summarize how learners interact with code puzzles?

We answer question 1) by comparing the performance of the group of users who used the static puzzle sequence with the group who used the adaptive puzzle sequence. We consider both puzzle completion rates and the M.E. experienced between the two groups. We conducted a Chi-squared test of homogeneity on the completion rates of the groups, and a Mann-Whitney U test comparing the average ME rating of each user for both groups.

We address question 2) by comparing self reported M.E. measurements of students who complete all tasks (26 *Finishers*) to those of students who drop out (42 *Quitters*). *Quitters*, in this case are those who drop out after having finished at least the DoT pathway. We compare the ME ratings by taking each user’s average ME for the DoT pathway and then conduct a Mann-Whitney U test across

the populations to test significant difference. Abandonment is high in online resources which introduce coding [18, 21, 36], producing a barrier to the online dissemination of coding skills. In this study, we consider the finishers to have had a successful experience, and the quitters to have had an unsuccessful experience.

Finally, we answer question 3) by parsing out high level interactions from user logs and compressing them into principal components that summarize each puzzle solving experience. We evaluate each component's relationship with cognitive load and success, to better understand common behaviours and their implications on learning.

4.1 Principal Components of Puzzle Solving

Principal Component Analysis (PCA) is widely used for dimensionality reduction in matrices by detecting correlations amongst the feature columns to identify underlying phenomena captured by the features. PCA has recently been used to discover and describe behavioural phenomena captured by user interactions within online library search sessions [4]. We use PCA similarly to [4] to gain insights into the common behaviors demonstrated in code puzzle solving experiences by 1) extracting features which summarize the user experience, 2) creating a matrix of each feature (columns) calculated for each puzzle attempted (rows), 3) transforming this matrix with Singular Value Decomposition [14] PCA using Scikit-Learn [28], and finally 4) examining the resulting PCA coefficients and feature correlations of the discovered components to reveal the most common behaviors of puzzle solvers.

4.1.1 Features. We parse user log files for 40 features designed to capture observed puzzle-solving interaction patterns. To discover relevant features, we used three methods. First, researchers watched several hundred puzzle solving sessions in preliminary user studies while taking field notes of behaviours and strategies. Second, we looked through the interaction log files for detectable patterns, and lastly we accounted for summary statistics. Two examples of high-level features are 1) **max-editing-in-a-row**: the maximum number of times a user manipulates the code blocks in their program without testing it or watching the correct animation and 2) **avg-incorrect-on-playmine**: the average number of code blocks that are not placed correctly when the user tests their code. All features are calculated per puzzle for each user.

4.1.2 Components. We calculate the aforementioned 40 features for 2057 total puzzles to create a 40 X 2057 feature matrix. Each feature column was then standardized into its z-score, and then we used PCA similarly to [4] to transform the normalized features, and examined the resulting PCA coefficients and feature correlations of these components to reveal common behaviors of puzzle solvers. We identify which behaviors may contribute to successful learning experiences, and describe how they may be useful to an adaptive code puzzle system. We present the first 7 principal components below.

1. Duration: The first component (C1) represents the duration of solving a code puzzle—correlating highly with features such as time spent and number of interactions. C1 also correlates highly with ME, which is intuitive because users who work on a problem for longer are likely struggling. This component is higher on average

in later code puzzles, and also is higher when users have more or longer bugs, and work slowly and inefficiently.

2. Understanding: Users prioritize information gathering and then make precise code insertions. C2 is a measurement of efficiency in code puzzle solving. This component correlates highly negatively with puzzle order, so this is likely more of an indicator of the simplicity of the puzzle rather than a strategy employed by successful learners.

3. Brute force: User edits code often without improvement. C3 suggests that a user is attempting to progress through the puzzle without understanding the content. This is a sign of disinterest, confusion, and ineffective learning. Students who attempt to “game” adaptive learning systems learn less than students who do not [5]. This component is useful and can be used to improve the code puzzle recommendation policy by holding back users who use brute force approach, and redirecting them to use information gathering strategies.

4. Editing to information gathering ratio: C4 is simply the ratio of editing to information gathering. This component correlates slightly negatively with ME and does not correlate with puzzle order seen, indicating that this has less to do with the puzzle and more to do with the user. Specifically, this may be higher for users who better understand the feedback provided by the animations.

5. Small picture iteration: User works on one small part of the puzzle at a time, making very few edits in a row without information gathering. C5 is high when a user inserts a block, tests their code, moves their block, and tests again iteratively. Since C5 does not correlate with puzzle order, it is likely revealing of a user's puzzle-solving strategy rather than a property of the puzzle itself.

6. Confusion: User reduces work space by removing blocks and works only on small pieces of the puzzle at a time. This can be useful for an online learning system because it indicates the user's need to gather information, which could be automatically suggested or played. Completed puzzles with high confusion ratings can also be a sign that a user should be held back in the adaptive pathway by being given a simpler puzzle next.

7. Knowledge seeking: Users move blocks around in chunks and prioritize reviewing their own animation in chunks. Higher C7 values indicates that the user tests code blocks often and thoroughly to understand what they do. This component shows that users are taking time to deeply understand what the code blocks do. By moving many blocks around before testing, the user may be mentally simulating the different code block configurations before attempting to verify their work. This may be a strategy which is germane to learning the relevant concepts at hand.

5 RESULTS

We address our findings for each of our questions. We highlight 1) learning in adaptive versus static pathways, 2) trends of cognitive load in successful learning experiences, and 3) how the first 7 principal components of code puzzle solving relate to success.

5.1 Adaptive Versus Static Puzzle Sequences

For the two groups, we found no significant difference between either attrition rates or reported cognitive load. The adaptive pathway

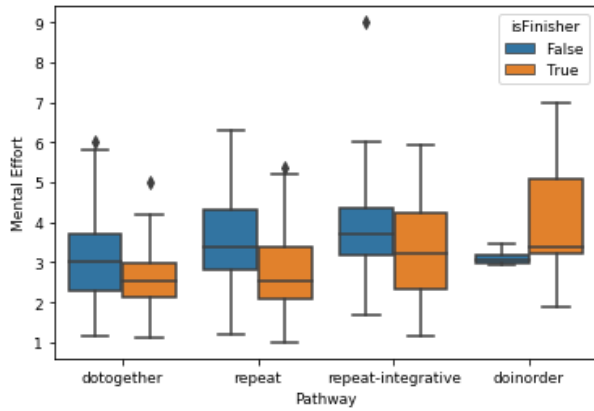


Figure 3: Box and whisker plots of user average ME by pathway: finishers vs quitters

users experienced slightly more cognitive load than the static pathway, but this difference is suggestive, and not statistically significant ($p=0.11$). This trend is not surprising, as both M.E. distributions are skewed to the left, and the trialled adaptive policy aims to push users away from the commonly experienced extreme low end of the cognitive load scale. Generally, the adaptive system pushed the users into more difficult puzzles more quickly.

5.2 High Cognitive Load is an Indicator for Dropout

We find that users who go on to finish all pathways follow a trend of lower average cognitive load ($M=3.2$, $std=1.0$) than those who eventually quit ($M=3.36$, $std=1.1$). This is generally consistent throughout all pathways. This data is shown in the box plots in Figure 3.

After a user completes the first full pathway (DoT), we show that their average experienced cognitive load can be used to predict eventual abandonment. A total of 68 users completed the first full (DoT) pathway. Of these users, 26 eventually completed all of the pathways, while the other 42 eventually abandoned their tasks.

We take the mean cognitive load readings for each user’s DoT puzzles and compare finishers to quitters using a Mann-Whitney U test; the results are shown in Table 1. We found that higher cognitive load early on is an indicator of dropout risk: users who drop out experience significantly ($p<0.05$) higher cognitive load ($M=3.22$, $std=1.13$) during the first full (DoT) pathway than those who will go on to complete all tasks ($M=2.59$, $std=0.88$). Going forward, we can consider this insight to provide users who are experiencing higher M.E. with supplementary or remedial puzzles before they progress to more difficult puzzles.

5.3 Principal Components of Code Puzzle Solving

Finally, we evaluate question 3) *What factors best summarize how learners interact with code puzzles, and how can these patterns be used to describe a puzzle-solving experience?*

Table 1: Mann-Whitney U test: quitters’ means vs. finishers’ averages during DoT pathway

Var	Finisher DoT Mean	Quitter DoT Mean	U stat (p)
C1	-1.16	-1.05	432 (0.076)
C2	0.7	0.62	543 (0.487)
C3	-0.17	-0.28	506 (0.309)
C4	0.17	0.16	519 (0.369)
C5	0	0.04	530 (0.422)
C6	-0.07	-0.02	489 (0.238)
C7	0.2	-0.01	327 (0.003)
ME	2.59	3.22	369 (0.013)

In this section, we address how the aforementioned 7 principal components relate to success or struggle. For each of the 7 discovered components, we take each user’s average across the first (DoT) pathway and compare users who eventually finish all tasks versus those who eventually quit. The group averages and results of Mann-Whitney U analyses for each of the components are in Table 1. For each analysis, the preconditions for a Mann-Whitney U test are satisfied: there are no dependencies within or between groups, the data is continuous, and each distribution shares a non-normal shape. Also, the Bonferonni correction is applied to ensure significance.

While most of these component DoT averages are similarly distributed between quitter and finisher groups, we note one main observation: component 7, *Knowledge Seeking*, is significantly higher ($p<0.005$) for users who go on to complete all tasks. By linking this strategy with success, we emphasize the need to design code puzzle interfaces to make it as easy and intuitive as possible for users to learn what the blocks do, and to consider this as a teachable strategy.

6 CONCLUSION AND DISCUSSION

Our results show that the trialled adaptive code puzzle system did not improve learner retention or impact demand on cognitive capacity. However, we did find that lower levels of cognitive load were indicative of successful learning experiences, and plan to leverage this finding to trial the next puzzle recommendation policy iteration to better avoid cognitive overload.

Since the trialled policy presented learners with more challenging tasks sooner, we plan to next trial a policy with more balanced recommendations, targeting M.E. ranges that are indicative of successful learning outcomes.

Our finding that ties successful learning experiences to lower cognitive load is consistent with CLT: learners who are experiencing non-extraneous cognitive load are still processing novel concepts, implying that they have yet to master the concepts themselves. The presence of heightened cognitive load is evidence of unlearned content the user has not yet mastered. Going forward, we recommend holding a user from the next pathway until they are easily solving the toughest puzzles of their current pathway. Our findings suggests testing a future puzzle selection policy which manages user M.E. closer to 2 (“very easy”) suggesting they have

some level of mastery of the subject without being unchallenged entirely.

7 LIMITATIONS

The predictive model we used to recommend future puzzles was trained on data of primarily young girls (and a few boys) learning to code. In our study, we deploy this model for both male and female learners. Since the populations of the training data and the deployment subjects are different, we cannot guarantee that the model yields the same accuracy. Another limitation is that because users completed code puzzle pathways remotely, we cannot guarantee that they did not collaborate or get help from others. However, this factor may be beneficial, since most online educational systems have no such guarantees, either.

REFERENCES

- [1] 2020. LightBot. <https://lightbot.com/>
- [2] 2020. Looking Glass. <https://lookingglass.wustl.edu/>
- [3] 2020. Scratch - Imagine, Program, Share. <https://scratch.mit.edu/>
- [4] Jaime Arguello and Anita Crescenzi. 2019. Using Principal Component Analysis to Better Understand Behavioral Measures and their Effects. In *Proceedings of the 2019 ACM SIGIR International Conference on Theory of Information Retrieval (ICITIR '19)*. Association for Computing Machinery, Santa Clara, CA, USA, 177–184. <https://doi.org/10.1145/3341981.3344222>
- [5] Ryan Shaun Baker, Albert T. Corbett, and Kenneth R. Koedinger. 2004. Detecting Student Misuse of Intelligent Tutoring Systems. In *Intelligent Tutoring Systems (Lecture Notes in Computer Science)*, James C. Lester, Rosa Maria Vicari, and Fábio Paragauçu (Eds.). Springer, Berlin, Heidelberg, 531–540. https://doi.org/10.1007/978-3-540-30139-4_50
- [6] Joseph E. Beck and Beverly Park Woolf. 2000. High-Level Student Modeling with Machine Learning. In *Intelligent Tutoring Systems (Lecture Notes in Computer Science)*, Gilles Gauthier, Claude Frasson, and Kurt VanLehn (Eds.). Springer, Berlin, Heidelberg, 584–593. https://doi.org/10.1007/3-540-45108-0_62
- [7] Klara Benda, Amy Bruckman, and Mark Guzdial. 2012. When Life and Learning Do Not Fit: Challenges of Workload and Communication in Introductory Computer Science Online. *ACM Transactions on Computing Education* 12, 4 (Nov. 2012), 15:1–15:38. <https://doi.org/10.1145/2382564.2382567>
- [8] Peter Brusilovsky, Charalampos Karagiannidis, and Demetrios Sampson. 2004. Layered evaluation of adaptive learning systems. *International Journal of Continuing Engineering Education and Lifelong Learning* 14, 4/5 (2004), 402. <https://doi.org/10.1504/IJCEELL.2004.005729>
- [9] Gino Camp, Fred Paas, Remy Rikers, and Jeroen van Merriënboer. 2001. Dynamic problem selection in air traffic control training: a comparison between performance, mental effort and mental efficiency. *Computers in Human Behavior* 17, 5 (Sept. 2001), 575–595. [https://doi.org/10.1016/S0747-5632\(01\)00028-0](https://doi.org/10.1016/S0747-5632(01)00028-0)
- [10] Ouhaio Chen, Juan C. Castro-Alonso, Fred Paas, and John Sweller. 2018. Extending Cognitive Load Theory to Incorporate Working Memory Resource Depletion: Evidence from the Spacing Effect. *Educational Psychology Review* 30, 2 (June 2018), 483–501. <https://doi.org/10.1007/s10648-017-9426-2>
- [11] Albert T. Corbett and John R. Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (Dec. 1994), 253–278. <https://doi.org/10.1007/BF01099821>
- [12] Barbara Ericson, Lauren E. Margulieux, and J. Rick. 2017. Solving parsons problems versus fixing and writing code. *Koli Calling* (2017). <https://doi.org/10.1145/3141880.3141895>
- [13] Stuart Garner. 2002. *Reducing the Cognitive Load on Novice Programmers*. Association for the Advancement of Computing in Education (AACE), P. <https://eric.ed.gov/?id=ED477013>
- [14] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. 2010. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *arXiv:0909.4061 [math]* (Dec. 2010). <http://arxiv.org/abs/0909.4061> arXiv: 0909.4061.
- [15] Kyle James Harms, Jason Chen, and Caitlin L. Kelleher. 2016. Distractors in Parsons Problems Decrease Learning Efficiency for Young Novice Programmers. In *Proceedings of the 2016 ACM Conference on International Computing Education Research (ICER '16)*. Association for Computing Machinery, Melbourne, VIC, Australia, 241–250. <https://doi.org/10.1145/2960310.2960314>
- [16] Kyle J. Harms, Noah Rowlett, and Caitlin Kelleher. 2015. Enabling independent learning of programming concepts through programming completion puzzles. In *2015 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 271–279. <https://doi.org/10.1109/VLHCC.2015.7357226>
- [17] Caitlin Kelleher and Wint Hnin. 2019. Predicting Cognitive Load in Future Code Puzzles. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300487>
- [18] René F. Kizilcec and Sherif Halawa. 2015. Attrition and Achievement Gaps in Online Learning. In *Proceedings of the Second (2015) ACM Conference on Learning @ Scale - L@S '15*. ACM Press, Vancouver, BC, Canada, 57–66. <https://doi.org/10.1145/2724660.2724680>
- [19] James A. Kulik and J. D. Fletcher. 2016. Effectiveness of Intelligent Tutoring Systems: A Meta-Analytic Review. *Review of Educational Research* 86, 1 (March 2016), 42–78. <https://doi.org/10.3102/0034654315581420> Publisher: American Educational Research Association.
- [20] Michael J. Lee and Andrew J. Ko. 2015. Comparing the Effectiveness of Online Learning Approaches on CS1 Learning Outcomes. In *Proceedings of the eleventh annual International Conference on International Computing Education Research (ICER '15)*. Association for Computing Machinery, New York, NY, USA, 237–246. <https://doi.org/10.1145/2787622.2787709>
- [21] Michael J. Lee, Andrew J. Ko, and Irwin Kwan. 2013. In-game assessments increase novice programmers' engagement and level completion speed. In *Proceedings of the ninth annual international ACM conference on International computing education research (ICER '13)*. Association for Computing Machinery, San Diego, San California, USA, 153–160. <https://doi.org/10.1145/2493394.2493410>
- [22] Richard E. Mayer and Roxana Moreno. 2003. Nine Ways to Reduce Cognitive Load in Multimedia Learning. *Educational Psychologist* 38, 1 (March 2003), 43–52. https://doi.org/10.1207/S15326985EP3801_6
- [23] Loredana Mihalcu, Ron J. C. M. Salden, Gemma Corbalan, Fred Paas, and Mircea Miclea. 2011. Effectiveness of cognitive-load based adaptive instruction in genetics education. *Computers in Human Behavior* 27, 1 (Jan. 2011), 82–88. <https://doi.org/10.1016/j.chb.2010.05.027>
- [24] Steven Oxman and William Wong. 2014. WHITE PAPER: ADAPTIVE LEARNING SYSTEMS. (2014), 30.
- [25] Fred Paas and John Sweller. 2012. An Evolutionary Upgrade of Cognitive Load Theory: Using the Human Motor System and Collaboration to Support the Learning of Complex Cognitive Tasks. *Educational Psychology Review* 24, 1 (March 2012), 27–45. <https://doi.org/10.1007/s10648-011-9179-2>
- [26] Fred Paas, Juhani E. Tuovinen, Huib Tabbers, and Pascal WM Van Gerven. 2003. Cognitive load measurement as a means to advance cognitive load theory. *Educational psychologist* 38, 1 (2003), 63–71.
- [27] Fred G. Paas. 1992. Training strategies for attaining transfer of problem-solving skill in statistics: A cognitive-load approach. *Journal of Educational Psychology* 84, 4 (1992), 429–434. <https://doi.org/10.1037/0022-0663.84.4.429> Place: US Publisher: American Psychological Association.
- [28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [29] Ron J.C.M. Salden, Fred Paas, Nick J. Broers, and Jeroen J.G. van Merriënboer. 2004. Mental Effort and Performance as Determinants for the Dynamic Selection of Learning Tasks in Air Traffic Control Training. *Instructional Science* 32, 1 (Jan. 2004), 153–172. <https://doi.org/10.1023/B:TRUC.0000021814.03996.ff>
- [30] Ingrid A. E. Spanjers, Pieter Wouters, Tamara van Gog, and Jeroen J. G. van Merriënboer. 2011. An expertise reversal effect of segmentation in learning from animated worked-out examples. *Computers in Human Behavior* 27, 1 (Jan. 2011), 46–52. <https://doi.org/10.1016/j.chb.2010.05.011>
- [31] John Sweller, Jeroen J. G. van Merriënboer, and Fred G. W. C. Paas. 1998. Cognitive Architecture and Instructional Design. *Educational Psychology Review* 10, 3 (Sept. 1998), 251–296. <https://doi.org/10.1023/A:1022193728205>
- [32] John Sweller, Jeroen J. G. van Merriënboer, and Fred Paas. 2019. Cognitive Architecture and Instructional Design: 20 Years Later. *Educational Psychology Review* 31, 2 (June 2019), 261–292. <https://doi.org/10.1007/s10648-019-09465-5>
- [33] Adam Szulewski, Andreas Gegenfurtner, Daniel W. Howes, Marco L. A. Sivilotti, and Jeroen J. G. van Merriënboer. 2017. Measuring physician cognitive load: validity evidence for a physiologic and psychometric tool. *Advances in Health Sciences Education* 22, 4 (Oct. 2017), 951–968. <https://doi.org/10.1007/s10459-016-9725-2>
- [34] Jeroen J. G. van Merriënboer and John Sweller. 2005. Cognitive Load Theory and Complex Learning: Recent Developments and Future Directions. *Educational Psychology Review* 17, 2 (June 2005), 147–177. <https://doi.org/10.1007/s10648-005-3951-0>
- [35] Bin Xie and Gavriel Salvendy. 2000. Prediction of Mental Workload in Single and Multiple Tasks Environments. *International Journal of Cognitive Ergonomics* 4, 3 (Sept. 2000), 213–242. https://doi.org/10.1207/S15327566IJCE0403_3 Publisher: Routledge _eprint: https://doi.org/10.1207/S15327566IJCE0403_3
- [36] An Yan, Michael J. Lee, and Andrew J. Ko. 2017. Predicting abandonment in online coding tutorials. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 191–199. <https://doi.org/10.1109/VLHCC.2017.8103467> ISSN: 1943-6106.